Week 1 – Microservices Infrastructure

Interview Q&A Cheat Sheet (Crisp & High-Impact)

This document lists **common interview questions** that can be asked based on Week 1 work, along with **concise, strong answers**. Answers are framed to show **system thinking**, not just implementation.

---

1. Why did you start with infrastructure instead of business logic?

Answer:

> I wanted to validate service boundaries, deployment, and inter-service communication first. In microservices, architectural mistakes are very costly to fix later. Once the infra is correct, feature development becomes safer and faster.

Signal: Risk awareness, senior mindset

---

2. What problem does the API Gateway solve?

Answer:

> The API Gateway acts as a single entry point to the system. It hides internal service topology, centralizes routing, and allows cross-cutting concerns like authentication, logging, and rate limiting to be handled in one place.

Signal: Correct gateway responsibility

---

3. Why not let frontend directly call backend services?

Answer:

> Direct calls tightly couple the frontend to backend topology. Using a gateway allows backend services to evolve independently without breaking the frontend and improves security by reducing exposed surfaces.

Signal: Decoupling & maintainability

---

4. Why Docker per service instead of one shared container?

Answer:

> Each service should own its runtime and dependencies. Docker-per-service enables independent deployment, scaling, and failure isolation, which is essential in microservices.

Signal: Deployment maturity

---

5. How do services discover each other?

Answer:

> Docker Compose provides built-in DNS. Services can communicate using service names as hostnames, which avoids hardcoded IPs and works well in local and containerized environments.

Signal: Practical Docker knowledge

---

6. How did you verify inter-service communication?

Answer:

> I executed HTTP calls from inside the API Gateway container to downstream services using their service names. Successfully hitting health endpoints confirmed networking and service discovery were correctly configured.

Signal: Validation, not assumptions

---

7. Why do all services expose a /health endpoint?

Answer:

> Health endpoints are required for operational readiness. They are used by load balancers and orchestration platforms to detect failures and restart unhealthy services.

Signal: Production awareness

---

8. What would break if a service goes down?

Answer:

> Only the functionality owned by that service would be impacted. Other services remain operational because there is no shared runtime or database.

Signal: Fault isolation

---

9. Why didn’t you add databases in Week 1?

Answer:

> Adding databases too early couples services prematurely. I wanted to validate communication and deployment first. Databases are introduced once service boundaries are stable.

Signal: Correct sequencing

---

10. Why is the frontend also a separate service?

Answer:

> It simulates a real production setup and enforces clean frontend-backend contracts. This also helps when implementing CORS, auth flows, and gateway routing later.

Signal: End-to-end thinking

---

11. What are the limitations of your current setup?

Answer:

> There’s no authentication, authorization, database, or observability yet. These were intentionally deferred to keep Week 1 focused on infrastructure correctness.

Signal: Honest trade-offs

---

12. How would this scale in production?

Answer:

The same structure maps well to Kubernetes. Docker Compose service names translate naturally to Kubernetes services, and the API Gateway pattern remains unchanged.

Signal: Production scalability

---

13. What did you gain by validating networking early?

Answer:

It eliminated an entire class of bugs related to service discovery, ports, and networking, which are otherwise hard to debug once business logic is added.

Signal: Engineering efficiency

---

14. One-line project explanation (must memorize)

"I first validated a production-style microservices infrastructure with Docker and an API Gateway, ensuring service isolation and communication before implementing any business logic."

How to Use This Sheet

- Revise before interviews
- Use answers as talking points, not scripts
- Start explanations from **architecture**, not features

Status: Ready for interviews